

Screenshot 1A

```
chaturya@debian:~/PES1UG24CS126-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
All Phase 1 tests passed.
```

Screenshot 1B

```
chaturya@debian:~/PES1UG24CS126-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
chaturya@debian:~/PES1UG24CS126-pes-vcs$
```

Screenshot 2A

```
chaturya@debian:~/PES1UG24CS126-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
chaturya@debian:~/PES1UG24CS126-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
```

All Phase 2 tests passed.

```
chaturya@debian:~/PES1UG24CS126-pes-vcs$
```

Screenshot 2B

```
chaturya@debian:~/PES1UG24CS126-pes-vcs$ find .pes/objects -type f
.pes/objects/e9/ebca50e86c0c08b0bcd3c4c646de0d340c5d72b8b275477aace3943c4d8281
.pes/objects/19/f5043660c808b7ed1db6a44b8025f0d17788eebeb15c19cd73cf4b7b6dc4c9
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
chaturya@debian:~/PES1UG24CS126-pes-vcs$ xxd .pes/objects/19/f5043660c808b7ed1db6a44b8025f0d17788eebeb15c19cd73cf4b7b6dc4c9 | head -20
00000000: 7472 6565 2034 3800 3130 3036 3434 2066  tree 48.100644 f
00000010: 696c 652e 7478 7400 0bd6 9098 bd9b 9cc5  ile.txt.....
00000020: 934a 610a b65d a429 b525 3611 47fa a7b5  .J[...].)%6.G...
00000030: b922 919e 9a23 143d  ."....#.=
chaturya@debian:~/PES1UG24CS126-pes-vcs$
```

Screenshot 3A

```
chaturya@debian:~/PES1UG24CS126-pes-vcs$ ./pes init
echo "test" > a.txt
./pes add a.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:    file.txt
  staged:    a.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: test_tree.c
  untracked: commit.h
  untracked: pes.h
  untracked: README.md
  untracked: Makefile
  untracked: object.c
  untracked: .gitignore
  untracked: index.h
  untracked: tree.h
  untracked: test_sequence.sh
  untracked: index.c
  untracked: pes.c
  untracked: tree.c
  untracked: test_objects.c
  untracked: commit.c
```

```
chaturya@debian:~/PES1UG24CS126-pes-vcs$ █
```

Screenshot 3B

```
chaturya@debian:~/PES1UG24CS126-pes-vcs$ cat .pes/index
100644 0bd69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d 1776658776 12 file.txt
100644 999f24152159e51756a944d32257bf22080ff8608fff87ca9a4a823764e13dbe 1776659003 5 a.txt
```

Screenshot 4A

```

chaturya@debian:~/PES1UG24CS126-pes-vcs$ rm -rf .pes

./pes init
echo "hello" > a.txt
./pes add a.txt
./pes commit -m "first"

echo "change" >> a.txt
./pes add a.txt
./pes commit -m "second"

echo "more" >> a.txt
./pes add a.txt
./pes commit -m "third"

./pes log
Initialized empty PES repository in .pes/
Committed: e5f87bb39259... first
Committed: 630688712d51... second
Committed: 6d3bb1c000cf... third
commit 6d3bb1c000cf6759c63c38604bd6424f1537c3170679bd128a8d8d172e502b9d
Author: PES User <pes@localhost>
Date: 1776659200

    third

commit 630688712d51536f01fa246cb034b43740ef01660a24bc7e3e2d9f3385d8686b
Author: PES User <pes@localhost>
Date: 1776659200

    second

commit e5f87bb39259ff4cd914e83bd89d7c1ce77293d3770935ba6c08fe64b5f078b3
Author: PES User <pes@localhost>
Date: 1776659200

    first92000
chaturya@debian:~/PES1UG24CS126-pes-vcs$ █

```

Screenshot 4B

```

chaturya@debian:~/PES1UG24CS126-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/02/c4d861b61beecd4064374a0ee731a9be527cc75e37e3b960356c480fdde356
.pes/objects/29/b2496335ba233a6637f78eda3b903cb11693cdc12f39eb08aa3a4961eb25c8
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/37/1aac45c3ba401d13c2ce98c9537b4e53c6049b950bf8a33cbc400712d9f4fa
.pes/objects/44/1f7bd1010846e6819318a7f377d78dbc3fffb46fd5836e3c5de55ae182cd9b7
.pes/objects/63/0688712d51536f01fa246cb034b43740ef01660a24bc7e3e2d9f3385d8686b
.pes/objects/6d/3bb1c000cf6759c63c38604bd6424f1537c3170679bd128a8d8d172e502b9d
.pes/objects/df/bf9e97e22e3aa20c4bc2d58cbcd31aa1f53547de56cc54d363adc5f64d647b
.pes/objects/e5/f87bb39259ff4cd914e83bd89d7c1ce77293d3770935ba6c08fe64b5f078b3
.pes/refs/heads/main
chaturya@debian:~/PES1UG24CS126-pes-vcs$

```

Screenshot 4C

```

chaturya@debian:~/PES1UG24CS126-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
6d3bb1c000cf6759c63c38604bd6424f1537c3170679bd128a8d8d172e502b9d
ref: refs/heads/main
chaturya@debian:~/PES1UG24CS126-pes-vcs$

```

FINAL INTEGRATION

```
UTM File Edit Virtual Machine View Window Help
Linux
Apr 20 09:59
chaturya@debian: ~/PES1UG24CS126-pes-vcs

chaturya@debian:~/PES1UG24CS126-pes-vcs$ make test-integration
=== Running integration tests ===
bash test sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
(nothing to show)

Untracked files:
(nothing to show)

--- First Commit ---
Committed: 31ccf69e48d3... Initial commit

Log after first commit:
commit 31ccf69e48d31e179d3c29475e1cbe6528126575c67ee897e20eb4d17658d424
Author: PES User <pes@localhost>
Date:   1776659363

    Initial commit

--- Second Commit ---
Committed: 323be1e0a9e4... Update file.txt

--- Third Commit ---
Committed: 602738aa46e2... Add farewell

--- Full History ---
```

```
UTM File Edit Virtual Machine View Window Help
Linux
Apr 20 10:00
chaturya@debian: ~/PES1UG24CS126-pes-vcs

--- Full History ---
commit 602738aa46e2746f97c8255d28775b13a155c41792d702de05d62e390f4e4dc2
Author: PES User <pes@localhost>
Date:   1776659363

    Add farewell

commit 323be1e0a9e449fed7ff807693a944e9ee2445589d3ee5249d254b6d08a6c177
Author: PES User <pes@localhost>
Date:   1776659363

    Update file.txt

commit 31ccf69e48d31e179d3c29475e1cbe6528126575c67ee897e20eb4d17658d424
Author: PES User <pes@localhost>
Date:   1776659363

    Initial commit

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
602738aa46e2746f97c8255d28775b13a155c41792d702de05d62e390f4e4dc2

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a1227423d119fd3c8d7c7216094ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/31/ccf69e48d31e179d3c29475e1cbe6528126575c67ee897e20eb4d17658d424
.pes/objects/32/3be1e0a9e449fed7ff807693a944e9ee2445589d3ee5249d254b6d08a6c177
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/60/2738aa46e2746f97c8255d28775b13a155c41792d702de05d62e390f4e4dc2
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b754be8c5542b562bf48bafaaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
chaturya@debian:~/PES1UG24CS126-pes-vcs$
```

Question 5.1

In the PES version control system, branching can be implemented by extending the reference mechanism used for HEAD.

Currently, the system stores the latest commit hash in `.pes/refs/heads/main` and the HEAD file points to this branch. To support branching, multiple branch files can be created under `.pes/refs/heads/`, where each file represents a branch and stores the commit hash of its latest commit.

Creating a new branch involves copying the current commit hash into a new file such as `.pes/refs/heads/feature`. Switching branches can be achieved by updating the HEAD file to point to the desired branch reference.

Since commits are immutable and identified by their hashes, multiple branches can safely point to the same commit and diverge independently as new commits are added. This design allows efficient branching without duplicating data.

Question 5.2

Merging in the PES version control system involves combining changes from two branches into a single commit. This can be achieved by identifying a common ancestor commit (base), and then comparing the changes in both branches relative to this base.

For each file, if changes are made in only one branch, the modified version can be directly included in the merged result. If both branches modify the same file differently, a merge conflict occurs. In such cases, the system can either insert conflict markers in the file or require manual resolution by the user.

After resolving differences, a new commit is created that includes the merged tree. This commit typically has two parent references, one for each branch being merged, allowing the history to reflect the merge operation.

This approach ensures that the commit history remains consistent while enabling collaborative development across multiple branches.

Question 5.3

The current PES implementation supports only a single branch (main) and does not include full branching functionality. It lacks features such as creating multiple branches, switching between branches, and maintaining separate commit histories.

To improve this, the system can be extended by storing multiple branch references under `.pes/refs/heads/`, with each branch pointing to its latest commit. A command for switching branches can update the HEAD reference accordingly.

Additionally, advanced features such as merge support, conflict resolution, and visualization of branch history can be implemented to make the system more robust and closer to real-world version control systems like Git.

These improvements would allow parallel development and better management of different versions of the project.

Question 6.1

Garbage collection in a version control system refers to the process of identifying and removing unused objects from the object store to save storage space.

In PES, objects such as blobs, trees, and commits are stored using content-addressable storage. Over time, some objects may become unreachable, meaning they are not referenced by any commit or branch.

Garbage collection works by starting from all reachable references (such as HEAD and branch pointers) and traversing the commit graph to mark all reachable objects. Any object not marked during this traversal is considered unused and can be safely deleted.

This process helps maintain efficiency by removing redundant or orphaned data while preserving all necessary history.

Question 6.2

Garbage collection in PES can be implemented using a mark-and-sweep approach.

First, the system identifies all reachable objects by starting from the HEAD reference and following all commit links, including parent commits and associated tree objects. Each tree object is further traversed to identify all referenced blob objects.

During this traversal, all reachable objects are marked. In the second phase, the system scans the `.pes/objects` directory and removes any object files that were not marked as reachable.

This ensures that only necessary objects are retained, while unused objects are deleted. Implementing this mechanism improves storage efficiency and keeps the repository clean without affecting valid commit history.